KOÇ UNIVERSITY

COMP 350
Introduction to DevOps

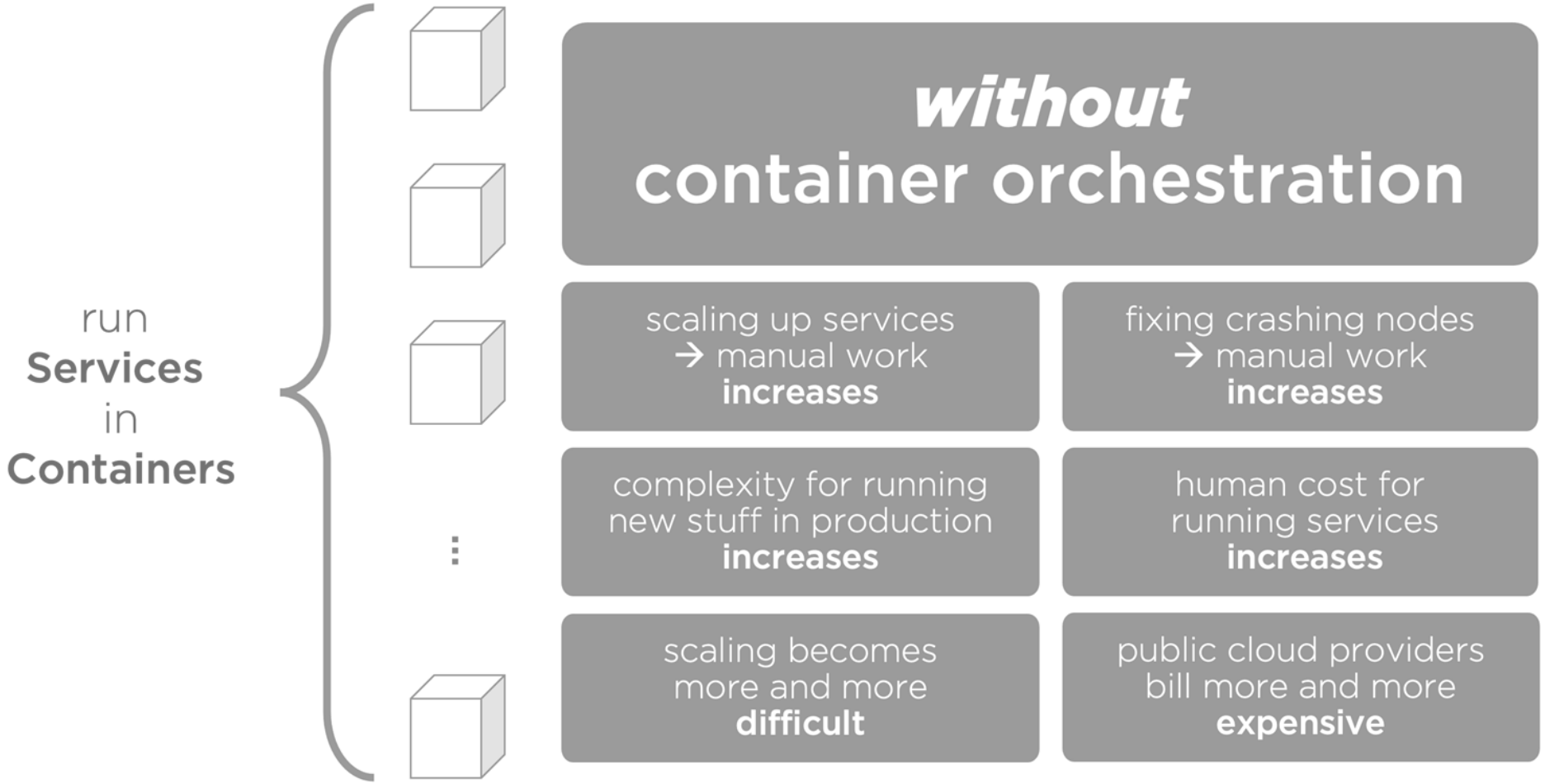
Lecture 10
Container Orchestration (Docker Swarm)

Hakan Ayral

What is container orchestration?

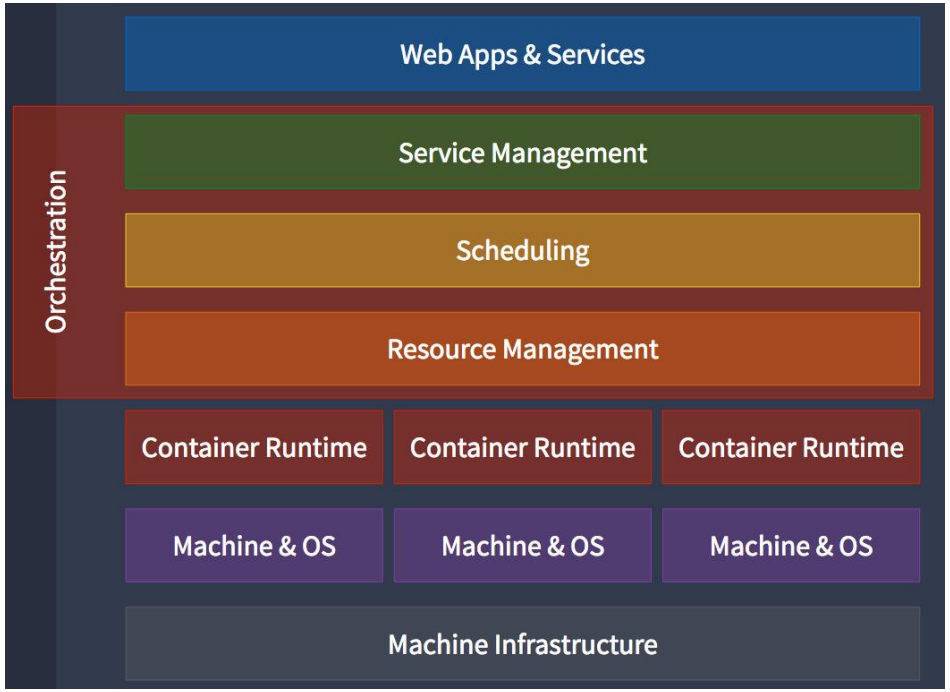
- Container orchestration is the automation of much of the operational effort required to run containerized workloads and services.
- This includes a wide range of things software teams need to manage a container's lifecycle, including:
  - provisioning,
  - deployment,
  - scaling (up and down),
  - networking,
  - load balancing and more.

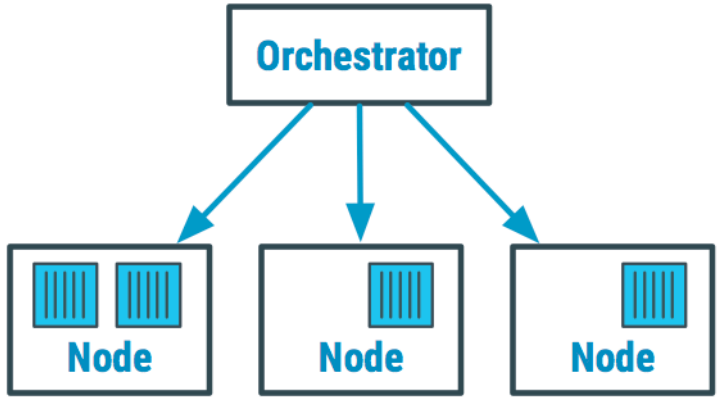
Containers Without Orchestration 👎



Container Orchestration

- Container orchestration automates and simplifies provisioning, and deployment and management of containerized applications.

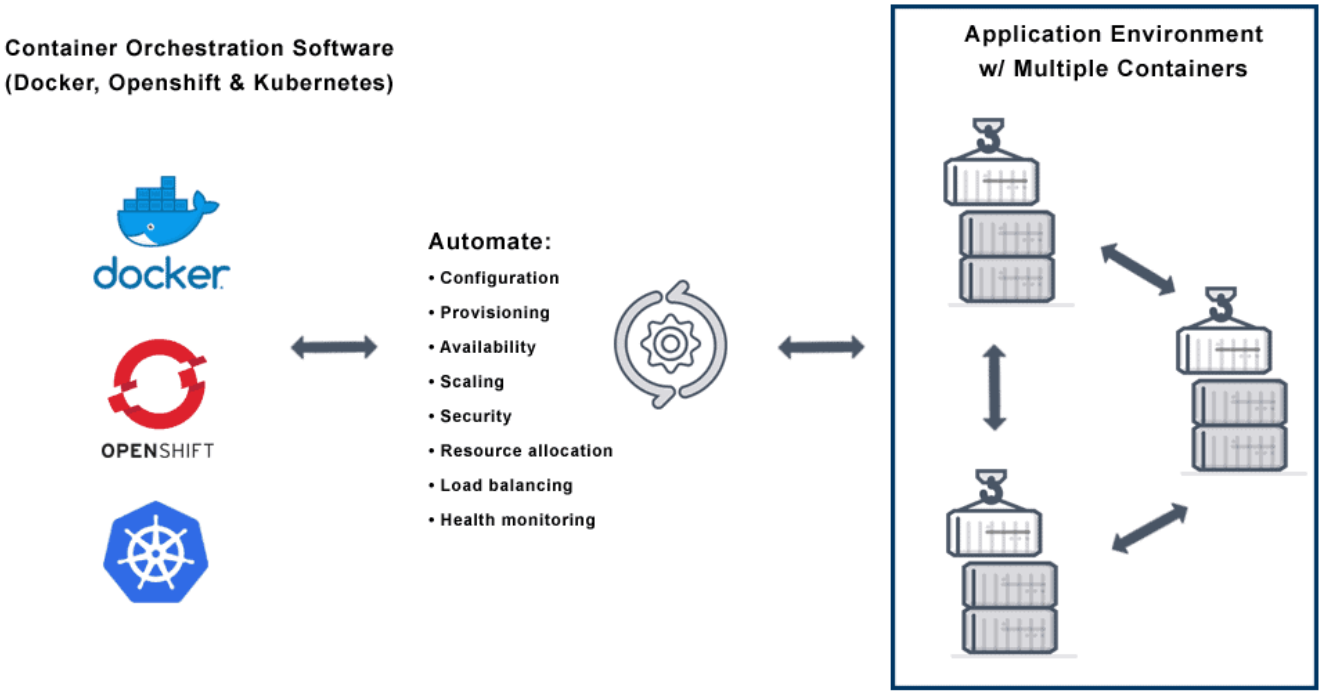




Container Orchestration

- Container orchestration is the automatic process of managing or scheduling the work of individual containers for applications based on microservices within multiple clusters.

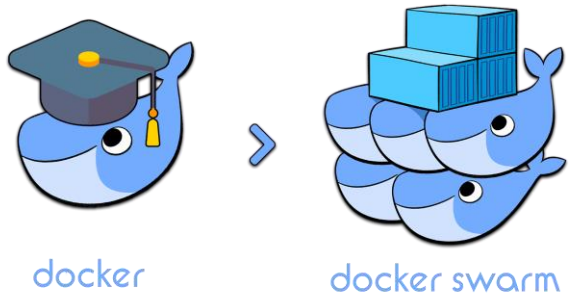
- The widely deployed container orchestration platforms are based on open-source versions like **Kubernetes**, **Docker Swarm** or the commercial version from Red Hat OpenShift.



Container Orchestration Benefits

- Fault-tolerance
- On-demand scalability
- Optimal resource usage
- Auto-discovery to automatically discover and communicate with each other
- Accessibility from the outside world
- Seamless updates/rollbacks without any downtime.

Docker Swarm



- What?
  - Swarm (called swarm mode) is a set of native Docker tools, which allow to deploy a set of Docker containers on multiple machines as if they were deployed on a single machine.

- Why?
  - Using only Docker requires a lot of error prone configuration and manual operations to boot and interconnect all the services.
  - Using virtual machines of heterogeneous size wastes a lot of unused memory and CPU capacity.
  - You may need horizontal scaling of specific services over multiple servers.
  - Other solutions, like Kubernetes, are too complex to setup, while having some limitations.

Docker Swarm

- Docker Swarm provides **native clustering functionality** for Docker containers, which turns a group of Docker engines into a single virtual Docker engine.
- The **`docker swarm`** CLI utility allows users to
  - run Swarm containers,
  - create discovery tokens,
  - list nodes in the cluster, and more.

```
hakan@hakan-VirtualBox:~$ docker swarm

Usage:  docker swarm COMMAND

Manage Swarm

Commands:
  init        Initialize a swarm
  join        Join a swarm as a node and/or manager
```

- The **`docker node`** CLI utility allows users to run various commands to manage nodes in a swarm, for example,
  - listing the nodes in a swarm,
  - updating nodes,
  - and removing nodes from the swarm.

```
hakan@hakan-VirtualBox:~$ docker node

Usage:  docker node COMMAND

Manage Swarm nodes

Commands:
  demote      Demote one or more nodes from manager in the swarm
  inspect     Display detailed information on one or more nodes
  ls          List nodes in the swarm
  promote     Promote one or more nodes to manager in the swarm
  ps          List tasks running on one or more nodes, defaults to current node
  rm          Remove one or more nodes from the swarm
  update      Update a node

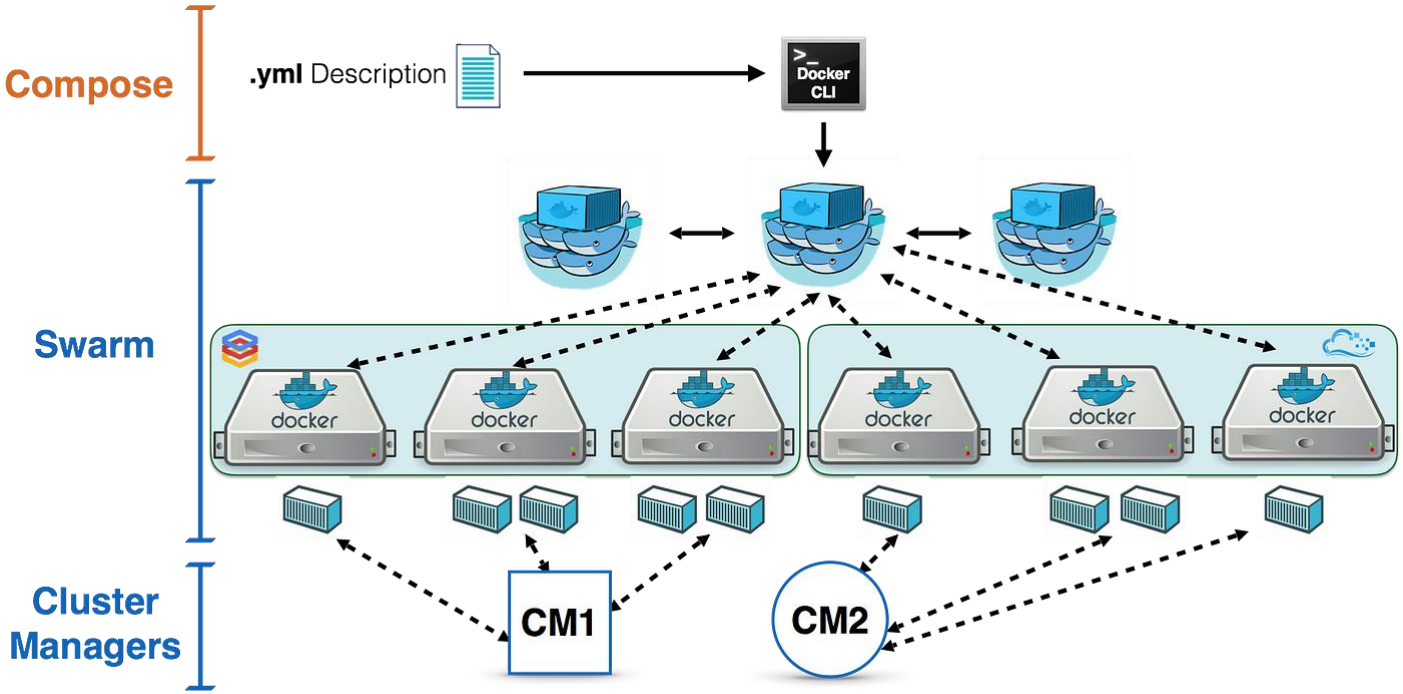
Run 'docker node COMMAND --help' for more information on a command.
```

- Docker manages swarms using the Raft consensus algorithm. According to Raft, for an update to be performed, the majority of Swarm nodes need to agree on the update.

Swarm Overview

- Multiple instances of a service can be replicated for load balancing and high availability
- The Transition from Docker Compose to swarm mode is trivial

- Key Concepts
  - **Swarm**
  - **Node**
  - **Service**
  - **Task**
  - **Load Balancing**



Key concepts - **Swarm**

- A **swarm** consists of multiple Docker hosts which run in Swarm mode and act as managers, to manage membership and delegation, and workers, which run swarm services.
  - A given Docker host can be a **manager**, a **worker**, or perform both roles.
  - When you create a service, you define its **optimal state** - number of replicas, network and storage resources available to it, ports the service exposes to the outside world, and more.
  - Docker works to maintain that desired state. For instance, if a worker node becomes unavailable, Docker schedules that node's tasks on other nodes.
    - A **task** is a running container which is part of a swarm service and is managed by a swarm manager, as opposed to a standalone container.

- One of the key advantages of swarm services over standalone containers is that **you can modify a service's configuration**, (*including the networks and volumes it is connected to*), **without the need to manually restart the service**.
  - Docker will update the configuration, stop the service tasks with out of date configuration, and create new ones matching the desired configuration.

- When Docker is running in Swarm mode, you can still run standalone containers on any of the Docker hosts participating in the swarm, as well as swarm services.
  - A key difference between standalone containers and swarm services is that only swarm managers can manage a swarm, while standalone containers can be started on any daemon.
  - Docker daemons can participate in a swarm as managers, workers, or both.

Key concepts - **Nodes**

- A **node** is an instance of the Docker engine participating in the swarm.
  - You can run one or more nodes on a single physical computer or cloud server, but production swarm deployments typically include Docker nodes distributed across multiple physical and cloud machines.

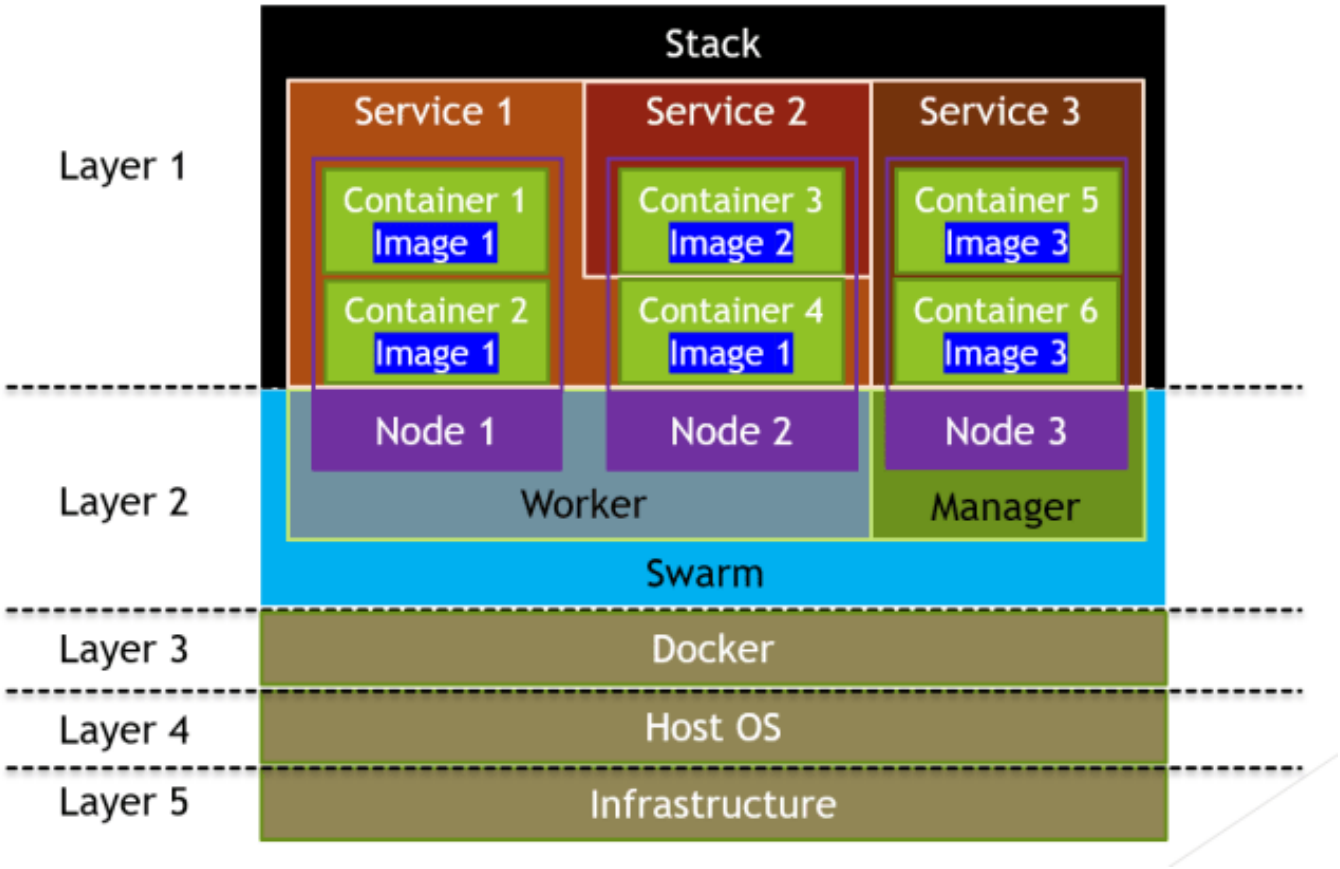
- To deploy your application to a swarm, you submit a **service definition** to a **manager node**.
  - The manager node dispatches units of work called **tasks** to worker nodes.
  - Manager nodes also perform the orchestration and cluster management functions required to maintain the desired state of the swarm.
  - Manager nodes elect a single leader to conduct orchestration tasks.

- **Worker nodes** receive and execute tasks dispatched from manager nodes.
  - Manager nodes also run services as worker nodes, but you can configure them to run manager tasks exclusively and be manager-only nodes.
  - An **agent** runs on each worker node and reports on the tasks assigned to it.
  - The worker node notifies the manager node of the current state of its assigned tasks so that the manager can maintain the desired state of each worker.

Key concepts – **Services** and **Tasks**

- A **service** is the definition of the tasks to execute on the manager or worker nodes.
  - It is the central structure of the swarm system and the primary root of user interaction with the swarm.
  - When you create a service, you specify which container image to use and which commands to execute inside running containers.
- In the replicated services model, the swarm manager distributes a specific number of replica tasks among the nodes based upon the scale you set in the desired state.
- For **global services**, the swarm runs one task for the service on every available node in the cluster.
- A **task** carries a Docker container and the commands to run inside the container.
  - It is the atomic scheduling unit of swarm.
  - Manager nodes assign tasks to worker nodes according to the number of replicas set in the service scale.
  - Once a task is assigned to a node, it cannot move to another node. It can only run on the assigned node or fail.

Nodes, Services and Swarm



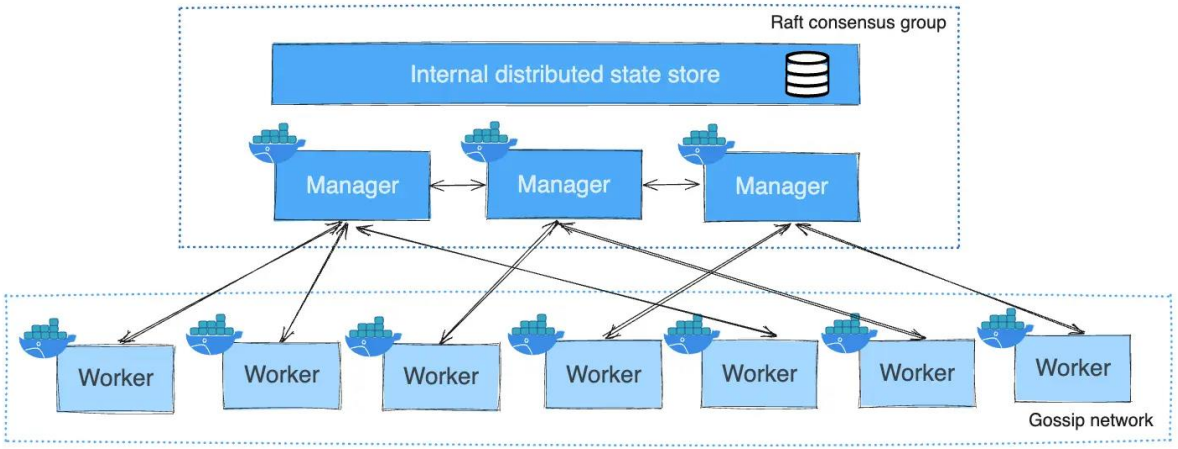
Key concepts – **Load Balancing**

- The swarm manager uses **ingress load balancing** to expose the services you want to make available externally to the swarm.
  - The swarm manager can automatically assign the service a published port or you can configure a published port for the service. You can specify any unused port. If you do not specify a port, the swarm manager assigns the service a port in the 30000-32767 range.

- External components, such as cloud load balancers, can access the service on the published port of any node in the cluster whether or not the node is currently running the task for the service.
  - All nodes in the swarm route ingress connections to a running task instance.

- Swarm mode has an **internal DNS** component that automatically assigns each service in the swarm a DNS entry.
  - The swarm manager uses **internal load balancing** to distribute requests among services within the cluster based upon the DNS name of the service.

How **manager nodes** work?

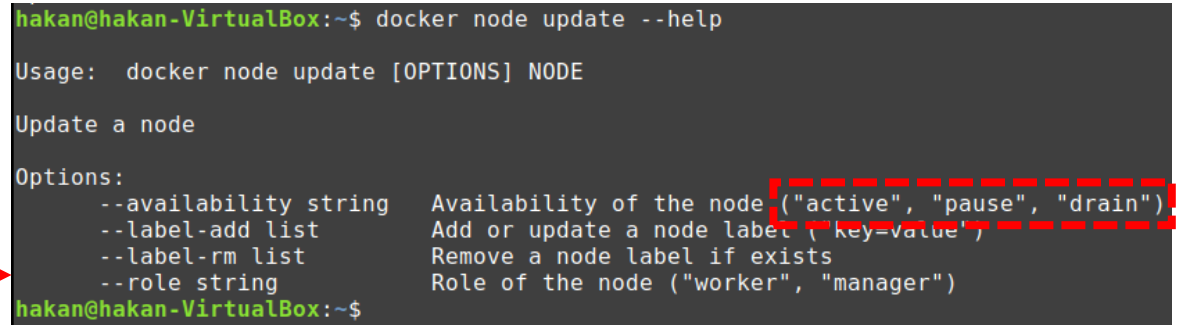


- Manager nodes handle cluster management tasks:
  - Maintaining cluster state
  - Scheduling services
  - Serving Swarm mode HTTP API endpoints
- Using a Raft implementation, the managers maintain a consistent internal state of the entire swarm and all the services running on it.
  - For testing purposes it is OK to run a swarm with a single manager.
  - If the manager in a single-manager swarm fails, your services continue to run, but you need to create a new cluster to recover.
- To take advantage of Swarm mode's fault-tolerance features, implement an **odd number of nodes**. When you have multiple managers you can recover from the failure of a manager node without downtime.
  - A 3-manager swarm tolerates a maximum loss of 1 manager.
  - A 5-manager swarm tolerates a maximum simultaneous loss of 2 manager nodes.
  - An odd number N of manager nodes in the cluster tolerates the loss of at most (N-1)/2 managers.
  - Docker recommends a maximum of **7** manager nodes for a swarm.

> ⚠ **Important**
>
> Adding more managers does NOT mean increased scalability or higher performance. In general, the opposite is true.

How **worker nodes** work?

- Worker nodes are also instances of Docker Engine whose sole purpose is to execute containers.
  - Worker nodes don't participate in the Raft distributed state, make scheduling decisions, or serve the swarm mode HTTP API.
- You can create a swarm of one manager node, but you cannot have a worker node without at least one manager node.
  - By default, all managers are also workers.
  - In a single manager node cluster, you can run commands like **docker service create** and the scheduler places all tasks on the local engine.
- To prevent the scheduler from placing tasks on a manager node in a multi-node swarm, set the availability for the manager node to **Drain**.
  - The scheduler gracefully stops tasks on nodes in **Drain** mode and schedules the tasks on an **Active** node.
  - The scheduler does not assign new tasks to nodes with **Drain** availability.
  - Use **docker node update** command to change node availability.



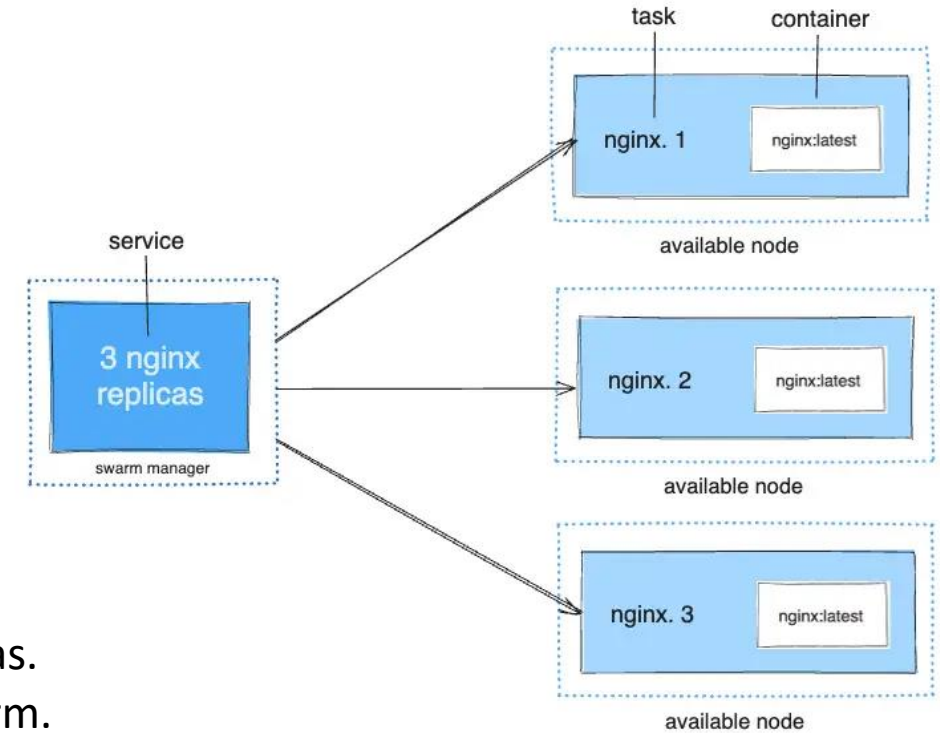
How **services** work?

- To deploy an application image when Docker Engine is in Swarm mode, you create a **service**.
  - Frequently a service is the image for a microservice within the context of some larger application.
  - Examples of services might include an HTTP server, a database, or any other type of executable program that you wish to run in a distributed environment.

- When you create a **service**, you specify which container image to use and which commands to execute inside running containers.
- You also define options for the **service** including:
  - The port where the swarm makes the service available outside the swarm
  - An overlay network for the service to connect to other services in the swarm
  - CPU and memory limits and reservations
  - A rolling update policy
  - The number of replicas of the image to run in the swarm

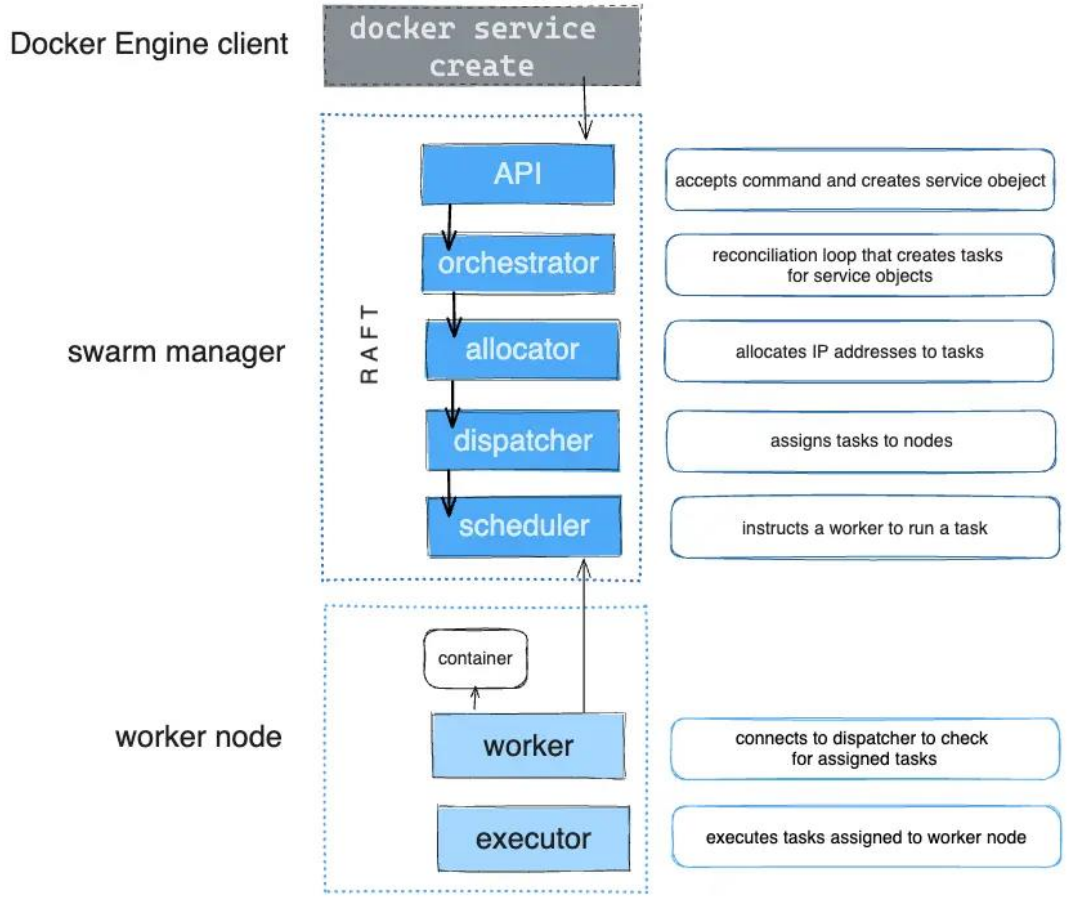
How **services** work?

- When you deploy the service to the swarm, the swarm manager accepts your service definition as the desired state for the service.
  - Then it schedules the service on nodes in the swarm as one or more replica tasks.
  - The tasks run independently of each other on nodes in the swarm.
- For example, imagine you want to load balance between three instances of an HTTP listener.
  - The diagram shows an HTTP listener service with three replicas.
  - Each of the three instances of the listener is a task in the swarm.
- A container is an isolated process.
  - In the Swarm mode model, each task invokes exactly one container.
  - A task is analogous to a “slot” where the scheduler places a container.
  - Once the container is live, the scheduler recognizes that the task is in a running state.
  - If the container fails health checks or terminates, the task terminates.



Tasks and scheduling

- A **task is the atomic unit of scheduling** within a swarm. When you declare a desired service state by creating or updating a service, the orchestrator realizes the desired state by scheduling tasks.
  - For instance, you define a service that instructs the orchestrator to keep three instances of an HTTP listener running at all times.
  - The orchestrator responds by creating three tasks. Each task is a slot that the scheduler fills by spawning a container.
  - The container is the instantiation of the task. If an HTTP listener task subsequently fails its health check or crashes, the orchestrator creates a new replica task that spawns a new container.
- **A task is a one-directional mechanism**. It progresses monotonically through a series of states: assigned → prepared → running → ...
  - If the task fails, the orchestrator removes the task and its container and then creates a new task to replace it according to the desired state specified by the service.



Swarm task states

- Docker lets you create services, which can start tasks. A service is a description of a desired state, and a task does the work. Work is scheduled on swarm nodes in this sequence:
  1. Create a service by using **docker service create**.
  2. The request goes to a Docker manager node.
  3. The Docker manager node schedules the service to run on particular nodes.
  4. Each service can start multiple tasks.
  5. Each task has a life cycle, with states like **NEW**, **PENDING**, and **COMPLETE**.
- Tasks are execution units that run once to completion.
  - When a task stops, it isn't executed again, but a new task may take its place.
- Tasks advance through a number of states until they complete or fail. Tasks are initialized in the **NEW** state. The task progresses forward through a number of states, and its state doesn't go backward. For example, a task never goes from **COMPLETE** to **RUNNING**.

Order of states **tasks** go through

Task state	Description
`NEW`	The task was initialized.
`PENDING`	Resources for the task were allocated.
`ASSIGNED`	Docker assigned the task to nodes.
`ACCEPTED`	The task was accepted by a worker node. If a worker node rejects the task, the state changes to `REJECTED`.
`READY`	The worker node is ready to start the task
`PREPARING`	Docker is preparing the task.
`STARTING`	Docker is starting the task.
`RUNNING`	The task is executing.
`COMPLETE`	The task exited without an error code.
`FAILED`	The task exited with an error code.
`SHUTDOWN`	Docker requested the task to shut down.
`REJECTED`	The worker node rejected the task.
`ORPHANED`	The node was down for too long.
`REMOVE`	The task is not terminal but the associated service was removed or scaled down.

How to **create** a swarm?

If you don't specify an advertise address, Docker checks if the system has a single IP address. If so, Docker uses the IP address with the listening port 2377 by default. If the system has multiple IP addresses, you must specify the correct --advertise-addr to enable inter-manager communication and overlay networking:

`docker swarm init --advertise-addr <MANAGER-IP>`

```
hakan@hakan-VirtualBox:~$ docker swarm init --advertise-addr 10.0.2.5
Swarm initialized: current node (9zhkyqg6ao5vlc3oe1hyzuec3) is now a manager.

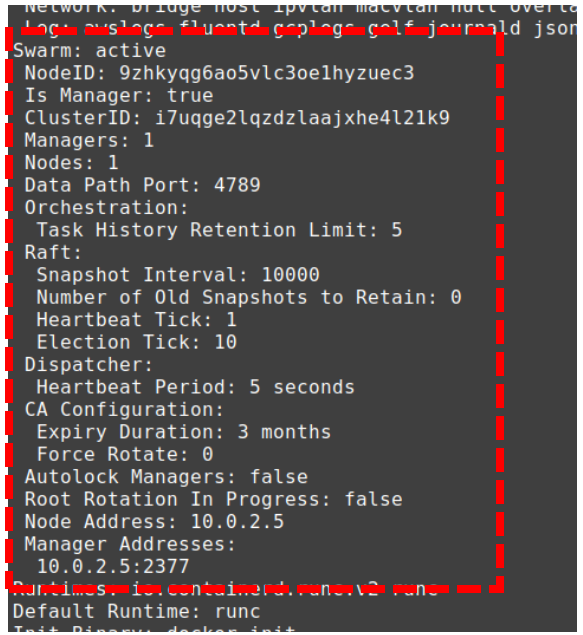
To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-4atnn59lqzzyibe2raw3in88y92kcu6fzxhs0g9oqq7h0f3pwq-7aiz7hqilzq5t5a9i3jrghc3a 10.0.2.5:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

hakan@hakan-VirtualBox:~$
```

- You can run **`docker info`** to view the current state of the swarm



- You can run the **`docker node ls`** command to view information about nodes
  - The * next to the node ID indicates that you're currently connected on this node.

```
hakan@hakan-VirtualBox:~$ docker node ls
ID                            HOSTNAME            STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
9zhkyqg6ao5vlc3oe1hyzuec3 *   hakan-VirtualBox    Ready     Active         Leader           25.0.4
cjje930ytaxus0053a5g8jlm7     hakan-VirtualBox    Ready     Active                          25.0.4
zorh46ls960210rp37onx1o39     hakan-VirtualBox    Ready     Active                          25.0.4
hakan@hakan-VirtualBox:~$
```

How to **add nodes** to swarm?

- Run the command shown at the output of the **`docker swarm init`** on the manager node to make a worker node join to the existing swarm

```
hakan@hakan-VirtualBox:~$ docker swarm init --advertise-addr 10.0.2.5
Swarm initialized: current node (9zhkyqg6ao5vlc3oe1hyzuec3) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-4atnn59lqzzyibe2raw3in88y92kcu6fzxhs0g9oqq7h0f3pwq-7aiz7hqilzq5t5a9i3jrghc3a 10.0.2.5:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

hakan@hakan-VirtualBox:~$
```

- If you don't have the command available, you can run the following command on a manager node to retrieve the join command for a worker: **`docker swarm join-token worker`**

How to **deploy** a service to the swarm?

```
hakan@hakan-VirtualBox:~$ docker service create --replicas 1 --name myservice alpine ping www.ku.edu.tr
nh680si1imrb4ms2tvxj3q63p
overall progress: 1 out of 1 tasks
1/1: running   [==================================================>]
verify: Service converged
hakan@hakan-VirtualBox:~$
```

- The **docker service create** command creates the service.
  - The **--name** flag names the service myservice.
  - The **--replicas** flag specifies the desired state of 1 running instance.
  - The arguments **alpine ping docker.com** define the service as an Alpine Linux container that executes the command ping docker.com.
- Run **docker service ls** to see the list of running services:

```
hakan@hakan-VirtualBox:~$ docker service ls
ID             NAME        MODE         REPLICAS   IMAGE          PORTS
nh680si1imrb   myservice   replicated   1/1        alpine:latest
hakan@hakan-VirtualBox:~$
```

How to **inspect** a service on swarm?

- Run **docker service inspect --pretty <SERVICE-ID>** to display the details about a service in an easily readable format.
  - To return the service details in json format, run the same command without the **--pretty** flag.

```
hakan@hakan-VirtualBox:~$ docker service inspect --pretty myservice

ID:             nh680si1imrb4ms2tvxj3q63p
Name:           myservice
Service Mode:   Replicated
 Replicas:      1
Placement:
UpdateConfig:
 Parallelism:   1
 On failure:    pause
 Monitoring Period: 5s
 Max failure ratio: 0
 Update order:      stop-first
RollbackConfig:
 Parallelism:   1
 On failure:    pause
 Monitoring Period: 5s
 Max failure ratio: 0
 Rollback order:    stop-first
ContainerSpec:
 Image:         alpine:latest@sha256:c5b1261d6d3e43071626931fc004f7014
 Args:          ping www.ku.edu.tr
 Init:          false
Resources:
Endpoint Mode:  vip

hakan@hakan-VirtualBox:~$
```

How to **inspect** a service on swarm?

- Run **docker service ps <SERVICE-ID>** to see which nodes are running the service
  - Swarm also shows you the **DESIRED STATE** and **CURRENT STATE** of the service task so you can see if tasks are running according to the service definition.

```
hakan@hakan-VirtualBox:~$ docker service ps myservice
ID             NAME          IMAGE          NODE              DESIRED STATE   CURRENT STATE           ERROR     PORTS
g0s0d8usixxu   myservice.1   alpine:latest   hakan-VirtualBox   Running         Running 6 minutes ago
hakan@hakan-VirtualBox:~$
```

- Run **docker ps** on the node where the task is running to see details about the container for the task.

```
hakan@hakan-VirtualBox:~$ docker ps
CONTAINER ID   IMAGE          COMMAND               CREATED         STATUS         PORTS     NAMES
6a125b130590   alpine:latest   "ping www.ku.edu.tr"   8 minutes ago   Up 8 minutes             myservice.1.g0s0d8usixxul6dr5leab4p3b
hakan@hakan-VirtualBox:~$
```

Manual **scaling**

- Run **`docker service scale <SERVICE-ID>=<NUMBER-OF-TASKS>`** to change the desired state of the service running in the swarm:

```
hakan@hakan-VirtualBox:~$ docker service scale myservice=2
myservice scaled to 2
overall progress: 2 out of 2 tasks
1/2: running   [==================================================>]
2/2: running   [==================================================>]
verify: Service converged
hakan@hakan-VirtualBox:~$
```

- Run **docker service ps <SERVICE-ID>** to see the updated task list:

```
hakan@hakan-VirtualBox:~$ docker service ps myservice
ID             NAME          IMAGE           NODE              DESIRED STATE   CURRENT STATE                ERROR     PORTS
g0s0d8usixxu   myservice.1   alpine:latest   hakan-VirtualBox   Running         Running 12 minutes ago
5daqkg9hybps   myservice.2   alpine:latest   hakan-VirtualBox   Running         Running about a minute ago
hakan@hakan-VirtualBox:~$
```

- Run **docker ps** to see the containers running on the node where you're connected.

How to **delete** a service on the swarm?

- Run **`docker service rm <SERVICE-ID>`** to remove a service.

```
hakan@hakan-VirtualBox:~$ docker service rm myservice
myservice
hakan@hakan-VirtualBox:~$
```

- You can use **`docker service inspect <SERVICE-ID>`** to verify that the swarm manager removed the service. The CLI returns a message that the service is not found.

```
hakan@hakan-VirtualBox:~$ docker service inspect myservice
[]
Status: Error: no such service: myservice, Code: 1
hakan@hakan-VirtualBox:~$
```

Change node roles

- You can promote a worker node to be a manager by running **docker node promote**.
  - For example, you may want to promote a worker node when you take a manager node offline for maintenance.

```
hakan@hakan-VirtualBox:~$ docker node ls
ID                            HOSTNAME           STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
9zhkyqg6ao5vlc3oe1hyzuec3 *   hakan-VirtualBox   Ready     Active         Leader           25.0.4
cjje930ytaxus0053a5g8jlm7     hakan-VirtualBox   Ready     Active                          25.0.4
zorh46ls960210rp37onx1o39     hakan-VirtualBox   Ready     Active                          25.0.4
hakan@hakan-VirtualBox:~$
hakan@hakan-VirtualBox:~$
hakan@hakan-VirtualBox:~$
hakan@hakan-VirtualBox:~$ docker node promote zorh46ls960210rp37onx1o39
Node zorh46ls960210rp37onx1o39 promoted to a manager in the swarm.
hakan@hakan-VirtualBox:~$
hakan@hakan-VirtualBox:~$ docker node ls
ID                            HOSTNAME           STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
9zhkyqg6ao5vlc3oe1hyzuec3 *   hakan-VirtualBox   Ready     Active         Leader           25.0.4
cjje930ytaxus0053a5g8jlm7     hakan-VirtualBox   Ready     Active                          25.0.4
zorh46ls960210rp37onx1o39     hakan-VirtualBox   Ready     Active         Reachable        25.0.4
hakan@hakan-VirtualBox:~$
```

- You can also demote a manager node to a worker node. See **docker node demote**.

Update a service

- You can change almost everything about an existing service using the docker service update command.
- When you update a service, Docker stops its containers and restarts them with the new configuration.
- Let's say you run Nginx as a service, and want to change the port you publish to clients outside the swarm to a different value (let's say to 80 from another number).
  - You can specify this when you create the service, using the -p.
  - When updating an existing service, the flag is --publish-add.
  - There is also a --publish-rm flag to remove a port that was previously published.

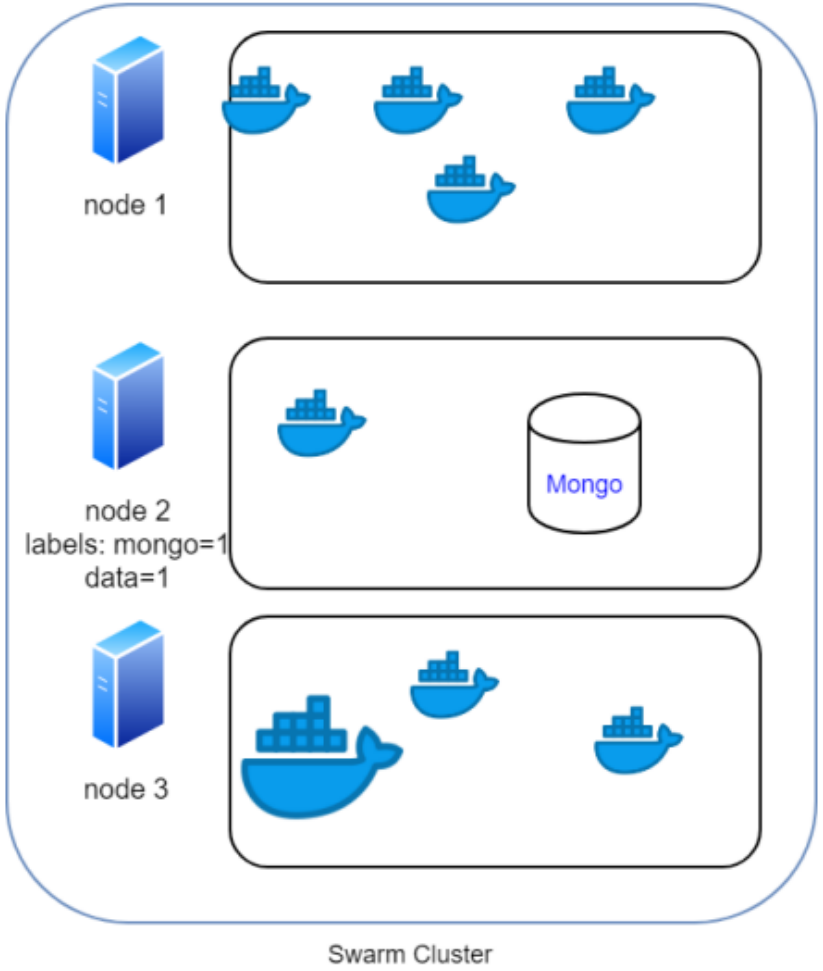
- Use the following command to update it to publish port 80

```
docker service update --publish-add 80 my_web_service
```

Swarm Differences: 1.Persistent Data

- **Events may cause the swarm to move a service to another node**
- Unfortunately, the persistent data won't be moved with it without volume plugins
- Workaround: use **node labels** and **placement rules** to make sure services with persistent data always stay on the same node:

```
services:
  mongo:
    deploy:
      placement:
        constraints:
          - node.labels.mongo == 1
```



Only use nodes which are labeled as **mongo**

Swarm Differences: 2.Resource Reservation

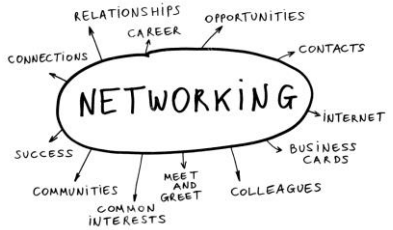
- docker-compose YAML format allows you to specify resources requirements
- The Docker Swarm container placement algorithm is greedy.
  - Make sure you specify large services not to be instanced on the same nodes where you previously pinned a service with persistent data.

```
services:
  space-time-tradeoff-software:
    deploy:
      resources:
        reservations:
          memory: 99999M
      placement:
        constraints:
          - node.labels.data != 1
```

Large service

Nodes reserved for services with persistent data

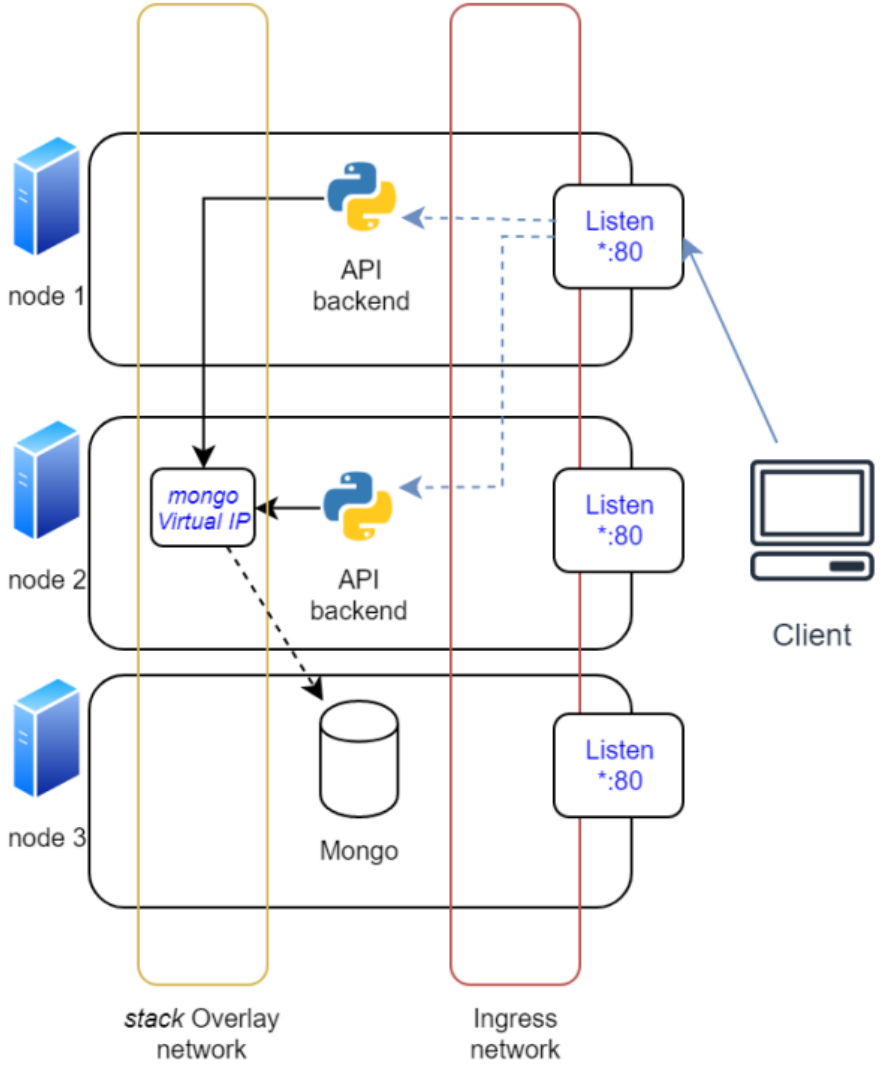
Swarm Differences: 3.Networking



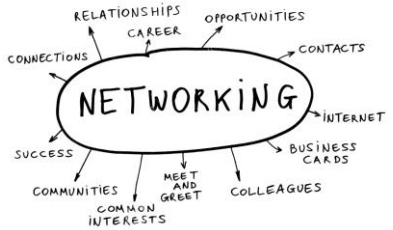
- Services within the network stack can locate other services by the hostname associated to a fixed Virtual IP within the stack **overlay network**.

- **Clients can connect to a service's published ports on any nodes of the swarm cluster**.
  - Requests are routed through the **ingress network**.

- Within stack and ingress networks, if a service is deployed with multiple replicas, incoming requests will be **load balanced**.



How does Docker-swarm networking work?

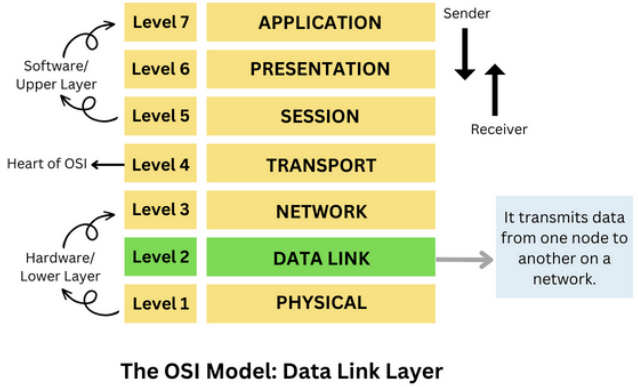


- We have already learned docker networking; but the scenario is not the same because here we are deploying docker containers on a swarm cluster.
- Docker provides three main types of networks that can be used to facilitate communication between containers:

  - **Bridge Network**
    - The bridge network is the default network that Docker creates when you install it.
    - Containers connected to the bridge network can communicate with each other using IP addresses.
    - By default, containers in the bridge network can communicate with each other, they can be exposed to the public by exposing a port.
    - Here we have a docker host port and container port mapping.
  - **Host Network**
  - **Overlay Network**

You see, this is different from the bridge networking of VirtualBox.
- On **VirtualBox** bridge means direct access to outer network.
- On **Swarm** bridge means containers on the same node can communicate with each other but not directly with the outer network.

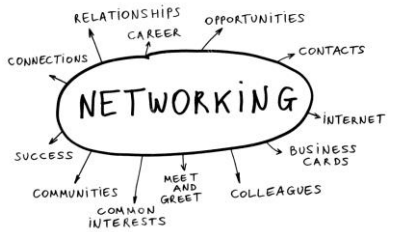
Docker **Bridge Network** details



The OSI Model: Data Link Layer

- In terms of networking, a bridge network is a **Link Layer** device which forwards traffic between network segments.
- In terms of Docker, a bridge network uses a software bridge which lets containers connected to the same bridge network communicate, while providing isolation from containers that aren't connected to that bridge network.
  - Docker bridge driver automatically installs rules in the host machine so that containers on different bridge networks can't communicate directly with each other.
- Bridge networks apply to containers running on the same Docker daemon host (i.e. same node).
  - For communication among containers running on different Docker daemon hosts, you can either manage routing at the OS level, or you can use an overlay network.
- When you start Docker, a default bridge network (named «**`bridge`**») is created automatically, and newly-started containers connect to it unless otherwise specified.
- You can also create user-defined custom bridge networks.
  - There are important differences between user-defined bridges and the default bridge.

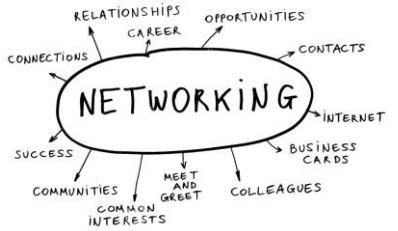
How does Docker-swarm networking work?



- We have already learned docker networking; but the scenario is not the same because here we are deploying docker containers on a swarm cluster.
- Docker provides three main types of networks that can be used to facilitate communication between containers:

  - **Bridge Network**
  - **Host Network**
    - Containers share the network namespace with the host machine.
    - This means containers bypass Docker's network isolation and directly use the host's network stack.
    - For example, if a web server is running inside a docker container using the host network uses port 80 of the host also.
    - This might cause the container ports to conflict with each other.
  - **Overlay Network**

Docker **Host Network** details



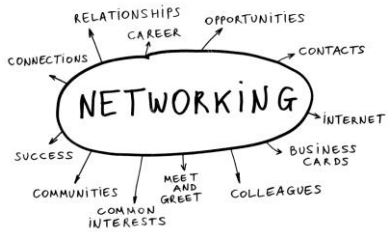
- If you use the host network mode for a container, that container's network stack isn't isolated from the Docker host (the container shares the host's networking namespace), and the **container doesn't get an IP-address of its own**.
  - For instance, if you run a container which binds to port 80 and you use host networking, the container's application is available on port 80 on the host's IP address.

- As the container does not have its own IP-address when using host mode networking, **port-mapping doesn't take effect, and the –p option is ignored**, producing the warning below:

WARNING: Published ports are discarded when using host network mode

- Host mode networking can be useful for optimizing performance, because it doesn't require network address translation (NAT), and no "userland-proxy" is created for each port.

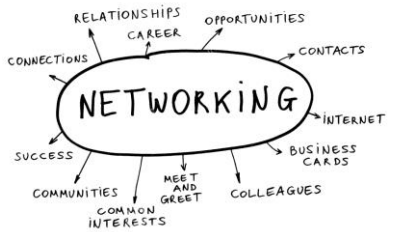
How does Docker-swarm networking work?



- We have already learned docker networking; but the scenario is not the same because here we are deploying docker containers on a swarm cluster.
- Docker provides three main types of networks that can be used to facilitate communication between containers:

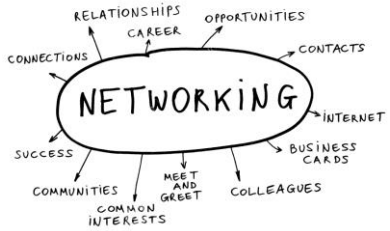
  - **Bridge Network**
  - **Host Network**
  - **Overlay Network**
    - Overlay networks are used for multi-host communication in Docker swarm mode.
    - They enable containers running on different hosts (nodes) to communicate with each other seamlessly as if they were on the same network.
    - By default Swarm mode uses a default address pool 10.0.0.0/8 for global scope (overlay) networks.

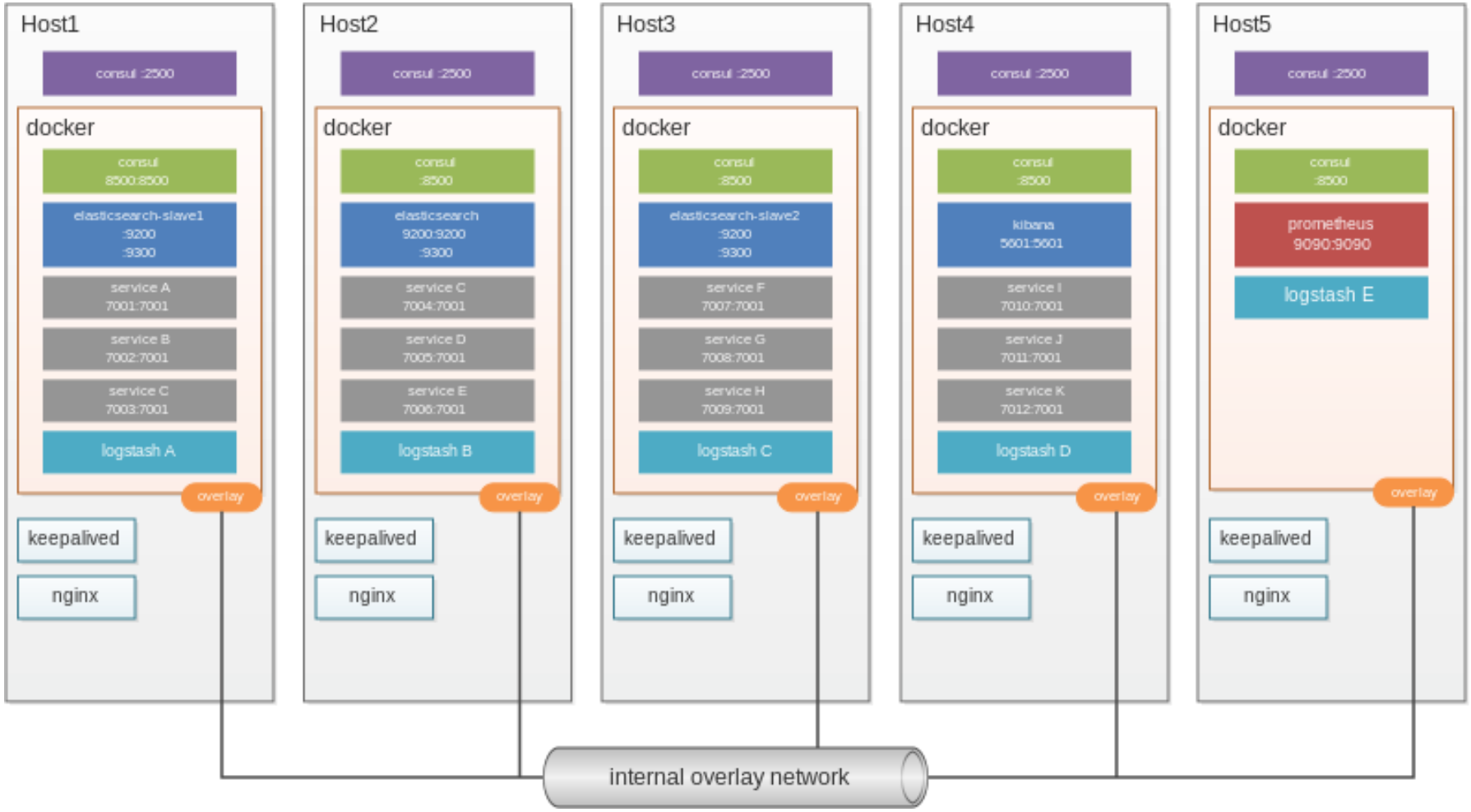
Docker **Overlay Network** details



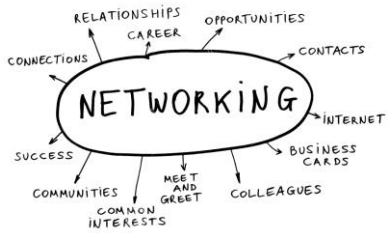
- The overlay network driver creates **a distributed network among multiple Docker nodes**.
  - This network sits on top of (overlays) the host-specific networks, allowing containers connected to it to communicate (*with option to encrypt traffic*).
  - Docker transparently handles routing of each packet to and from the correct Docker node and the correct destination container.
- You can create **user-defined overlay networks** using **`docker network create`**, in the same way that you can create user-defined bridge networks.
  - Services or containers can be connected to more than one network at a time.
  - Services or containers can only communicate across networks they're each connected to.
- Overlay networks are often used to create a connection between Swarm services, but you can also use it to connect standalone containers running on different hosts.
  - When using standalone containers, **it's still required that you use Swarm mode** to establish a connection between the hosts.

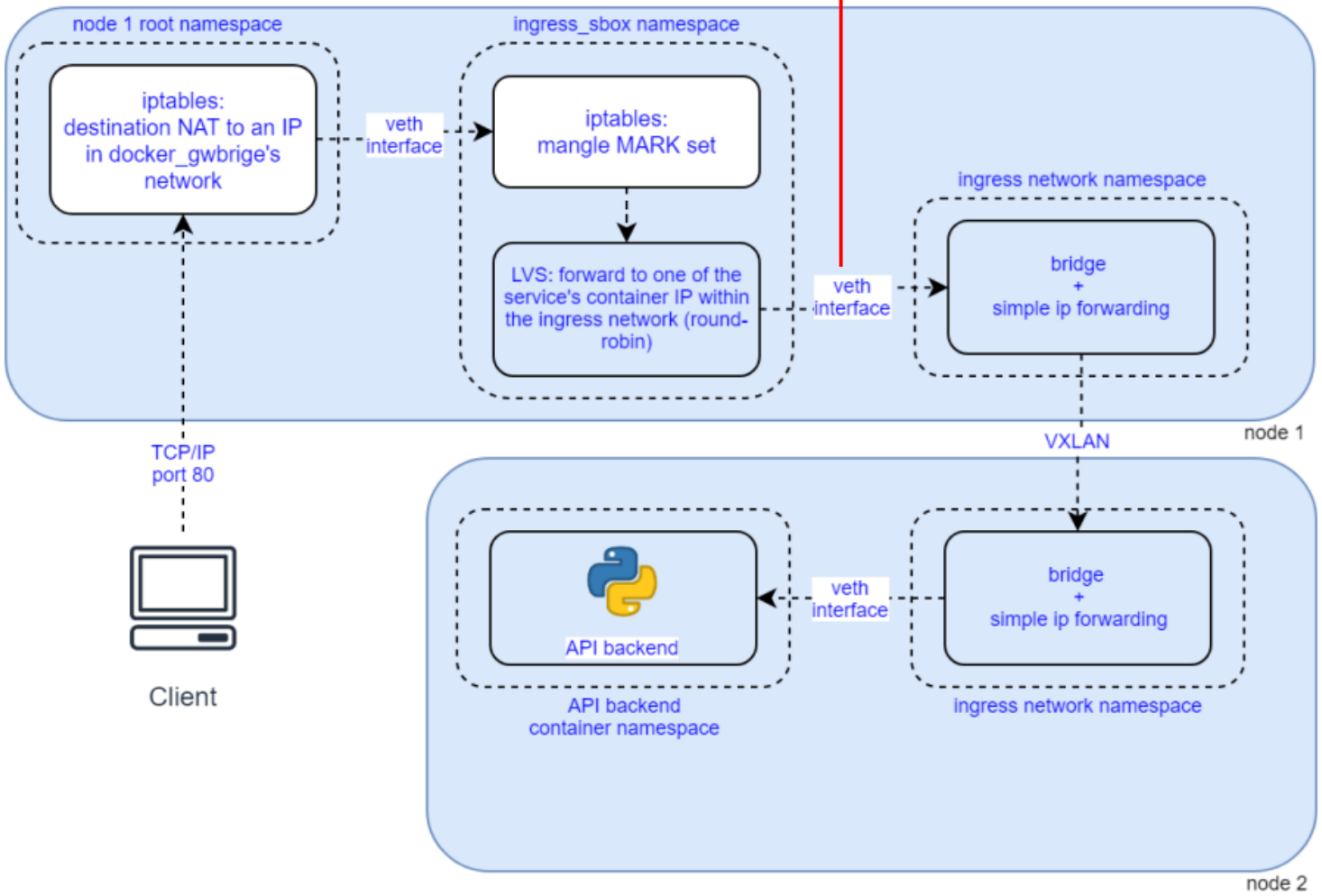
Docker Overlay Network





Ingress Network Under the Hood






```
veth(4)                    Kernel Interfaces Manual                    veth(4)

NAME                                                                   top

       veth - Virtual Ethernet Device

DESCRIPTION                                                            top

       The veth devices are virtual Ethernet devices.  They can act as
       tunnels between network namespaces to create a bridge to a
       physical network device in another namespace, but can also be
       used as standalone network devices.

       veth devices are always created in interconnected pairs.  A pair
       can be created using the command:

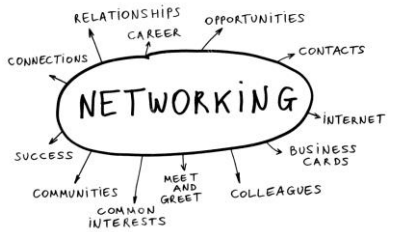
           # ip link add <p1-name> type veth peer name <p2-name>

       In the above, p1-name and p2-name are the names assigned to the
       two connected end points.

       Packets transmitted on one device in the pair are immediately
       received on the other device.  When either device is down, the
       link state of the pair is down.

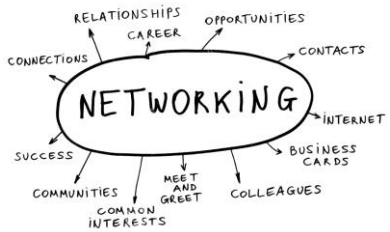
       veth device pairs are useful for combining the network facilities
       of the kernel together in interesting ways.  A particularly
       interesting use case is to place one end of a veth pair in one
       network namespace and the other end in another network namespace,
       thus allowing communication between network namespaces.  To do
```

Potential network pitfalls with swarm mode



- Where it can go wrong:
  - **Network namespaces**: Docker's way of isolating container networking
  - **IP forwarding & bridging**: forwards data from a network interface to another
  - **iptables**: Linux's way of implementing firewall rules
  - **LVS** (ipvsadm): virtual server and load balancing at kernel level
  - **Virtual ethernet interfaces** (veth): interfaces link between network namespaces
  - **VXLAN**: virtual network distributed over swarm nodes, using UDP transport

Symptoms of some common Networking Problems

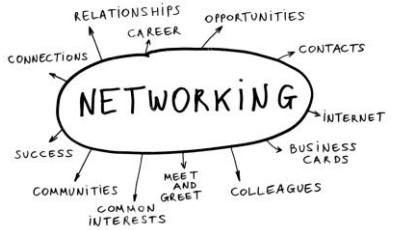


- While a project may work perfectly using only Docker Compose, running in swarm mode, you can experience:
  - Random requests time out to databases, especially when a connection pool is being used
  - HTTP requests to stack services with more than 900 seconds before the response will never get answered

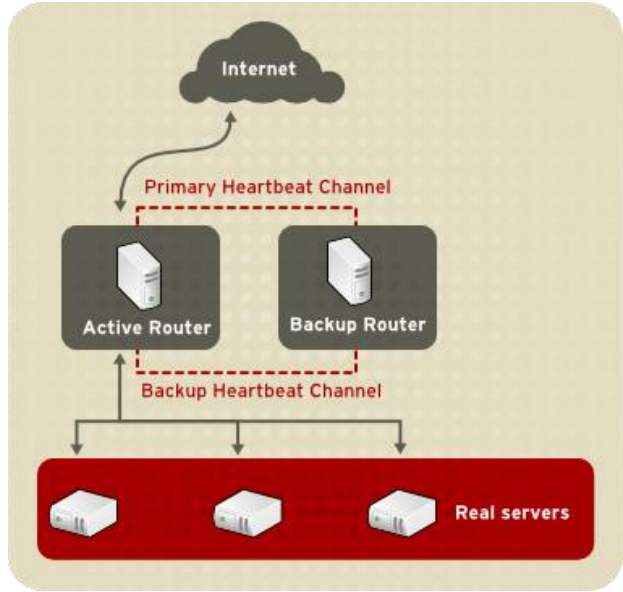


```
@app.route('/')
def hello_world():
    time.sleep(905)
    return 'Hello, World!'
```

The Culprit: Linux Virtual Server (LVS)

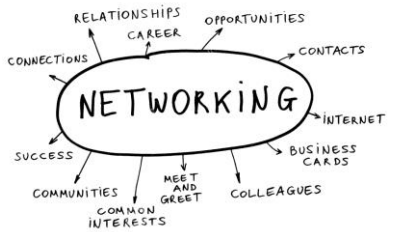


- Linux Virtual Server (LVS) is the **load balancing** software integrated into the OS for Linux kernel–based operating systems.
  - The major work of the LVS project is to develop
    - advanced **IP load balancing** software (IPVS),
    - **application-level load balancing** software (KTCPVS),
    - and cluster management components.
  - LVS can be used for building highly scalable and highly available network services.
  - The LVS components depend upon the Linux Netfilter framework.
    - LVS can handle UDP, TCP layer-4 protocols as well as FTP passive connection by inspecting layer-7 packets.



- Features:
  - No userspace software required (it is all in kernel)
  - Allows load-balancing of a virtual service to multiple real servers
  - **Incorporated in Linux kernel source tree since 2.4.x**

The Culprit:
Linux Virtual Server (LVS)



- Problems:
    - By default, **connection entries expire after 15 minutes of inactivity**
    - When a connection is expired, **no TCP finalize or reset is sent**
    - Subsequent packets within a TCP connection will be silently dropped (!)
- Solution/workaround:
    - Lower the **sysctl net.ipv4.tcp_keepalive_time** to 600 seconds (it's 2 hours by default): this will reset the LVS timeout every 10 minutes.
    - Cons: keepalive is not enabled by default: it's a socket option.